

Behind the Streams: Exploring Twitch's Technical Design

Vivian Kieuanh Nguyen

University of California, Santa Cruz

CSE 185S: Technical Writing for Computer Science and Engineering

Instructor Gerald Moulds

December 11, 2024

Behind the Streams: Exploring Twitch's Technical Design

Live streaming has transformed our consumption of content and entertainment, creating an entirely new category of digital engagement. Platforms supporting this feature empower creators to share activities and connect with audiences in real-time, no matter where they are. Twitch.tv, a pioneering live-streaming platform, has popularized this medium for diverse audiences and communities, offering streams centered on gaming, music, discussions, art, and more. The unparalleled level of engagement and interactivity between creators and their audiences has made live streaming one of the most sought-after forms of digital entertainment in recent years.

Twitch stands out as a leader in the live-streaming industry by integrating advanced server systems with machine-learning algorithms to deliver seamless, real-time content. Its core technologies enable the platform to:

1. Provide personalized watching recommendations tailored to individual users.
2. Minimize latency and enable real-time interactions between creators and viewers.
3. Support millions of global users, both as content creators and audiences.

By combining innovative technology with a focus on user experience and community-building, Twitch has become the benchmark for live-streaming platforms. This guide discusses the systems powering Twitch's success, offering invaluable insights into how cutting-edge technology and thoughtful user experience design come together to create a highly personalized, engaging, and scalable ecosystem. Twitch serves as a blueprint for understanding the broader live-streaming industry, demonstrating how sophisticated infrastructure and a user-centric approach can create a thriving platform for both creators and viewers.

This guide caters to three distinct audiences: beginners seeking to understand the fundamentals of live-streaming technology, streamers looking to optimize their content delivery, and tech enthusiasts interested in Twitch's innovative systems and methodologies. With Twitch as a primary example, the guide highlights the specialized technical requirements of live-streaming platforms and their real-world applications.

Through an exploration of topics such as real-time streaming, recommendation algorithms, and quality optimization, this guide offers a thorough overview designed to inform and inspire every audience. Readers will gain foundational knowledge of how Twitch has crafted seamless and personalized user experiences, paving the way for a new era of digital interactivity.

Real-Time Streaming Mechanics

At the core of Twitch's performance success lies its ability to deliver high-quality, low latency video and audio content to millions of users around the world. Using real-time streaming mechanics, it is possible to create live seamless broadcasts, creating an interactive environment where viewers and streamers can connect instantaneously.

Server Flow

A *server flow* is a structured series of processes where data is transmitted from a source—in this case, the streamer—to the end user, the viewer. It comprises the technical stages of data handling, from encoding the initial video feed to delivering it in a playable format. Twitch's server flow is developed in a way that ensures that live streams are seamlessly uploaded through the integration of advanced technologies prioritizing speed, quality, and reliability.

Twitch's server flow involves several key technologies that work together to manage video and audio data with efficiency.

What is a protocol? Before understanding what the Real-Time Messaging Protocol (RTMP) is, it is important to have an understanding of what a protocol is in the context of web development. A *protocol* can be compared to a rule that helps computers and devices communicate with each other while sharing information. Protocols follow standard instructions that will determine how pieces of information are meant to be sent and received. Having these established protocols is important because not all information is of the same type and will have specific requirements for different devices. Protocols help to organize the process of sending and receiving data into clear steps:

1. **Breaking Down Data:** Any data being transported needs to be divided into even smaller pieces known as packets.

2. Providing Instructions: Each packet is paired with instructions specifying:

- a. Destination: Where the packet is to be sent.
- b. Source: Where the packet originates.
- c. Reassembly Instructions: How the packets should be placed back in order.

Assembly is necessary because packets are not sent to achieve efficient delivery.

3. Sending Packets: Packets travel through the internet, passing through servers, then routers, as instructed to. Their small size allows for the traveling to be much faster.

4. Receiving Packets: The destination device reads the packet's information, reassembles it in proper order, and follows the directions as to what to do with the whole piece of data.

A protocol can be compared to a scenario in which two people who cannot speak the same language are trying to hold a conversation. The protocol would be a translation app that allows these individuals to understand what the other person is conveying without confusion. Protocols ensure devices can send, receive, and interpret, even if the original data is sent out in a different format.

Real-Time Messaging Protocol (RTMP). *The Real-Time Messaging Protocol (RTMP)*, establishes a persistent connection that transfers ongoing live video and audio directly from the streamer's device to Twitch's servers. The moment the user begins recording, this protocol breaks down pieces of data into smaller fragments to be sent quickly over the internet almost instantaneously. Having this protocol message optimized reduces the amount of delays, also known as *low latency*, so the timing of the stream delivered to the server is meant to be instantaneous as it occurs. RTMP is used with streaming software and devices, making it a widely utilized protocol that all live streaming platforms use.

How RTMP works:

1. Establishing a Connection: RTMP creates a stable connection linking the streamer's device to Twitch's servers. The connection is maintained with a common protocol called the *Transmission Control Protocol (TCP)* which makes sure the delivery is reliable and stable.
2. Breaking Down Data: The video and audio information is broken down into small packets, each containing necessary information such as timestamps and sequence numbers to ensure that everything is synchronized.
3. Transmitting Packets: The packets are sent quickly over the internet, ensuring that the content is sent to the servers in near real-time.
4. Receiving and Preparing for Playback: Once Twitch's servers receive the RTMP data, they begin to convert it into other formats that will be viewable for watchers, such as HTTP Live Streaming (HLS).

Routing RTMP. According to an article by Twitch's video ingest team: Rohit Puri, Christopher Lafata, Kevin Pan, and Eric Kwong (2022), RTMP works alongside Twitch's *Points of Presence (POPs)*, which are Twitch's 100 servers placed globally, paired with routing technology to maintain seamless stream delivery. These POPs are the first point of contact for streamers, allowing the data to be sent to the server to be processed in the most efficient and fastest way possible by routing to the physically closest data processing server.

Twitch created *Intelligest*, a routing system that helps enhance and manage incoming RTMP streams. Intelligest was made to maximize the use of network resources with real-time responses to traffic spikes and system failures (Puri et al. 2022).

Key features of Intelligest:

1. **Dynamic Routing:** Intelligent sends RTMP streams from POPs to data centers according to real-time server capacity and ongoing traffic in the network. It works to find the best solution in an efficient manner.
2. **Real-Time Response to Traffic Spikes:** Popular streams or ongoing events cause traffic surges, which incur Intelligent to re-adjust and restore balance.
3. **Failure Mechanisms:** Intelligent will reroute streams to alternative solutions in the event that a data center runs out of capacity or the system fails in order to maintain the streaming service for the creators and watchers.
4. **Optimization of Resources:** Intelligent makes optimal decisions to maximize efficiency and minimize operational costs.

HTTP Live Streaming (HLS). Once Twitch's server receives the data transferred by RTMP, it relies on the *HTTP Live Streaming (HLS)* protocol to reorganize and make the video data accessible to a large range of users. HLS is designed to optimize playback by adapting the *video quality resolutions* (e.g., 480p, 720p, 1080p, 1440p). This means that the viewable video is automatically adjusted according to the internet connection of the device. A slower internet connection will result in lower resolutions, but the HLS adjustment keeps viewers connected to the stream, allowing them to enjoy uninterrupted content. For visualization purposes, imagine a streaming water faucet with inconsistent water pressure. With the water pressure, the flow of water can be strong or weak but it continues to run steadily as it is able to.

How HLS works:

1. **Breaking Down Data:** After the video data is reassembled by Twitch's servers through RTMP, HLS will now divide it into small *segments* of just a few seconds of content. Each

segment is copied into different resolution types to accommodate differing devices and speeds.

2. **Adaptive Bitrate Streaming:** HLS automatically adjusts the video quality based on the viewer's current internet connection. The quality adjustment prevents *buffering*, the term for when viewers are completely disconnected from the content as the platform attempts to reconnect in search of faster internet speeds.
3. **Compatibility Across Devices:** As long as the device is capable of receiving and supporting web protocols, HLS can work across a wide range of device types, from smartphones to computers.

Content Delivery Networks (CDNs). Ensuring that video streams are delivered efficiently and reliably, Twitch uses *Content Delivery Networks (CDNs)* which are global networks of servers that strategically reduce latency and improve performance. When viewers click on a stream, the content is sent from the nearest Twitch server in the user's local region. The closer and faster content is sent to viewers, the more minimized delays and buffering occurrences will be. With the support of CDNs as well as Twitch's global network, Twitch is able to handle millions of ongoing streams at the same time.

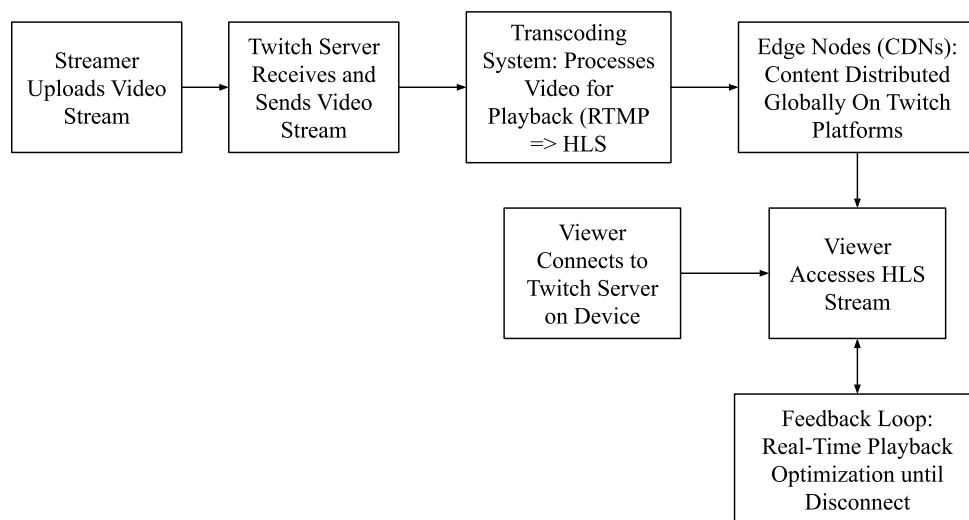
Playback. *Playback* is the final step in Twitch's server flow in which the segmented video data delivered by CDNs are finally decoded and displayed to the viewer's device. At this point, the CDNs send many short video segments to the watcher's device, which will reassemble them back together to create a full and consistent stream. The Playback on the device decodes the data into audio and video formats that are most compatible with the viewer's device. Playback helps to synchronize the stream with the other features on Twitch. such as the real-time

chat and interactive overlays that appear on the screen. Playback is the key feature that makes the stream visible and organized to the audience.

The full picture. The diagram below illustrates the whole server flow, explaining how data is transferred from the moment a creator begins their stream recording to the viewer accessing the stream.

Figure 1

Real-Time Server Flow



Note. This figure is an organized visualization generalizing how data is processed during a live video stream.

The flow of data ensures that viewers can watch with minimal latency and interruptions, no matter their location or type of device. Beginning from the streamer's upload with RTMP, the video data is processed into segments using HLS, distributed globally by CDNs, and finally decoded for playback on the audience's devices. Real-time chat operates alongside this system, ensuring consistent and interactive engagement throughout the stream.

Real-Time Chat Systems

In addition to streaming video and audio, Twitch has real-time chat systems synchronized with streams, playing a crucial role in encouraging viewers to interact with streamers. Chats help to foster community engagement and help streamers build a strong connection with their audience. Twitch relies on the *WebSocket Protocol* which allows for live communication streams from the watcher's chatting, sent directly to Twitch's servers. As such, viewers can send and receive messages from other viewers nearly instantaneously as their connection is kept open until disconnecting from the stream. Before a message is officially broadcasted, Twitch has moderation features, one being *AutoMod*, which detects potentially harmful or inappropriate content and allows users with admin access to manually approve or disprove. If appropriate, the *PubSub* (Publish-Subscribe) system broadcasts live updates on new chat messages, subscription notices, and donations, allowing all viewers to see when anyone sends a message in the chat.

Recommendation Algorithms

A *Recommendation Algorithm* is an algorithm designed to connect users on a platform with content that aligns with their interests. It is integrated into most social media platforms, including Twitch. Twitch's recommendation system attempts to dynamically analyze user behavior to deliver well-tailored suggestions of streams, categories, and channels that users may enjoy. These systems aim to increase user engagement with the platform and improve discoverability for content creators.

Input Variables

According to Twitch's Legal Terms of Service (2024), the inputs that impact what the platform will recommend to users include:

1. Follows: These are equivalent to “likes” on other media platforms. This input is a direct indication that a viewer is interested in a specific channel or category.
2. Watch History: Tracks streams and creators previously watched.
3. Search History: Captures the content users actively look for.
4. Engagement Metrics: Tracks the duration and frequency of interactions with previous streams watched.
5. Preference Survey Response: A recent feature that directly surveys the interests of new accounts created on the platform. This preference survey is optional but gives the platform an immediate understanding of what the user plans to watch.

Using these inputs, Twitch uses a machine learning algorithm to actively read through the collected input data to produce various kinds of recommendations. On the homepage, users can see direct recommendations of ongoing streams, specific categories, and channel suggestions in

the navigation sidebar. Users can fine-tune these recommendations through a designated “not interested button” to remove any suggestions that do not fit their interests.

Weighing inputs: A small-scale study observing the impact of user interactions. To analyze how quickly Twitch’s recommendation system adapts to user preferences, a small-scale experiment was conducted. Five accounts with distinct input activities were created, each designed to test specific input types and their impact on the algorithm’s responsiveness. The study aimed to observe the number of trials necessary to reflect a targeted category not featured on the original untailored homepage and if any input type had more influence.

Study design. Each account simulated specific types of user interaction inputs:

1. Account 1 (Follows Only): Without answering the preference survey, followed specific channels and categories directly showcasing the targeted game category.
2. Account 2 (Watch Only): Without answering the preference survey, watched streams at different lengths of time without any interactions.
3. Account 3 (Search Only): Without answering the preference survey, only active and direct searches related to the category were done.
4. Account 4 (All Inputs with Fine-Tuning): Without answering the preference survey, all inputs of following, watching, and searching with fine-tuning were done.
5. Account 5 (Survey Response Only): Immediately answered the preference survey at the initial opening of a new account with no further interactions.

Procedure. Before the preference application, the homepage features the highest trending creators and categories. The study relied on manual observations noting changes per every turn on the level of adjustment to the homepage from the beginning homepage. One turn is equivalent to executing the action and returning to the front homepage afterward.

In the timing of the study, the targeted less popular category is named “APEX Legends” (APEX) and a notable popular category was “World of Warcraft” (WoW). Through the execution of the study, lowering the dominance of WoW and seeing direct recommendations of APEX was a signal that the algorithm was taking in the user inputs.

Measurements. Homepage adjustments were evaluated using four points of scoring:

1. “0”: The homepage has remained the same as the original and WoW is dominant.
2. “0.5”: The homepage is gravitating towards APEX by recommending similar genre categories with a reduction in WoW dominance.
3. “0.75”: The homepage recommends some streams showing APEX and channels featuring APEX.
4. “1”: The homepage directly recommends APEX in categories, channels, and ongoing streams.

Results. The observational record of each account was ended once recommendations of APEX were clear on the homepage or discontinued due to clear weakness in weight. The study investigating the weight of each input revealed that the algorithm does not have a steady progression toward the desired category. Adjustments often fluctuate and temporarily gravitate toward similar categories. Potential flaws with this study come from the manual recording of observations.

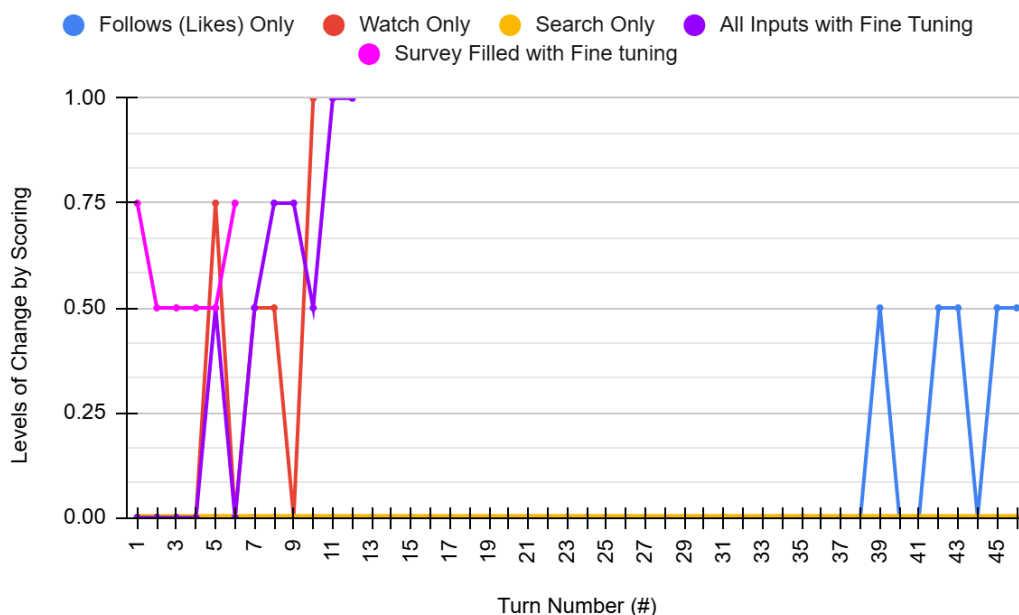
Key findings by account: A summary of notable discoveries during the experiment.

1. Follows Only: After the 42nd turn, the platform enacted a restriction that prevented the continued interaction. Only following channels streaming the targeted category, the algorithm did not recommend any content or channels related to APEX, only showing more popular trending content occasionally with similar genres to APEX at most.

2. Watch Only: After the fourth turn of different intervals of watching, an APEX channel was recommended momentarily, disappearing afterward. The homepage feed returned to its original state before showing similar genres for two more turns and returning. An intermission of eight hours was taken before returning to the account, after which the entire homepage had transformed and directly recommended APEX, dominating all sections.
3. Search Only: Searching was deemed to be the lowest weighing input, never altering the homepage even after intermissions were taken.
4. All with Fine-Tuning: Fine-tuning yielded the fastest results without intermissions compared to the other inputs that did not use the survey. By the fourth turn, APEX began to appear on the homepage and steadily increased in frequency of appearances. By the 12th turn, APEX was directly recommended through categories and channels.
5. Initial Survey: Right after answering the survey, the homepage began with recommendations of APEX in ongoing streams and channels but was not yet directly mentioned as its category. The results reveal the highest impact in time on the homepage compared to any other input forms.

Figure 2

Recommendation Model Input Weight Observation Results



Note. This is a line chart graph representing the intensity of changes as the recommendation algorithm continued to update throughout various trials.

Discussion. The algorithm requires multiple levels of engagement to produce accurately personalized results. Adjustments were not linear and often included temporary shifts to and from related genres. A notable observation reveals a hidden bias within the algorithm in which the platform will continually show popular content despite the user's fine-tuning against specific content. Questions may be raised regarding the platform's potential bias toward trending categories and potentially sponsored content, the balance between user needs, and company revenue goals. The delayed adaptation and promotion of non-relevant content may be challenging to viewers looking for niche categories, as well as creators promoting non-trending content. Discovering the reliance on large data sets highlights the necessity of transparency over recommendation generation for any platform.

Seamless Stream Quality Optimization

Efficient video compression and latency management are critical for Twitch and other streaming platforms to deliver high-quality, low-latency content to users worldwide. These technologies work to create seamless experiences through the optimization of data and minimizing of delays.

Understanding Compression and Latency Management

Compression techniques. *Compression techniques* optimally transmit video data by reducing the size of the file while preserving the resolution quality. This method allows Twitch to support millions of streams occurring simultaneously.

A key portion of this process involves managing *bandwidth*, which is the amount of data that can be sent over an internet connection during a specific period. Higher-quality resolution videos will require more bandwidth, which leads to delays and buffering for viewers with slower internet speeds. Compression works to ensure that streams are adaptable for any speed of the internet; where even the user with the slowest of connections will experience smooth, high-quality stream playbacks.

Latency management: Delivering real-time interactivity. *Latency* refers to the delay between an action made during a live video feed and its appearance on the viewer's screen. Achieving a minimized latency is critical for making real-time interactions possible, something central to the live-streaming experience. On a platform made for live streaming, interactive elements such as the live chat system, polls, interactive overlays, and other features that allow engagement with the streamer call for a high demand of low latency.

Understanding latency in the context of streaming. Latency is relevant during multiple portions of the server flow (see Figure 1), from the streamer uploading to Twitch's server,

server-side processing, and from the content delivery by CDNs to the viewer's device. On a platform made for live streaming, interactive elements like live chats, polls, and intensive streams, have a high demand to be low latency. Twitch employs several techniques and technologies to effectively manage and reduce latency:

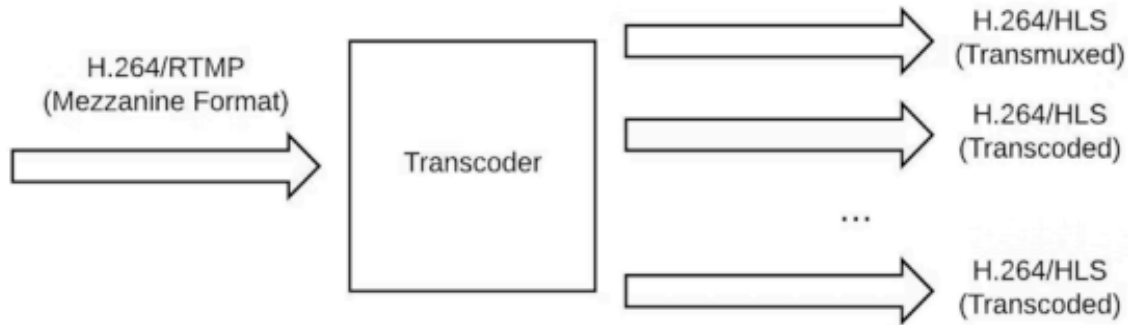
1. Optimizing RTMP and HLS protocols.
2. Optimizing CDNs.
3. *Bitrate Optimization*: Dynamically adjusts video quality based on internet speeds.
4. *Edge Nodes*: The use of local servers nearest to viewers, reducing the data transmission times and handling network traffic.
5. Adaptive Bitrate Streaming.
6. Efficient Encoding/Decoding: Optimized with the TwitchTranscoder, video encoding is done at significantly high speeds with low latency.

TwitchTranscoder: Twitch's Custom Compression Model

A majority of media platforms rely on FFmpeg, a software project handling compression for general use. To handle the higher demands of live streaming, Twitch developed the TwitchTranscoder, a custom software solution that outperforms FFmpeg to handle high volumes of live streaming activities. The TwitchTranscoder employs the *H.264 Codec* to balance quality retention and video compressions. The *codec*, a software that processes these compression transformations, reduces file sizes for efficient data transmission without significant quality loss, making it ideal for live streaming.

Figure 3

Visualization of the Transcoder Input and Output



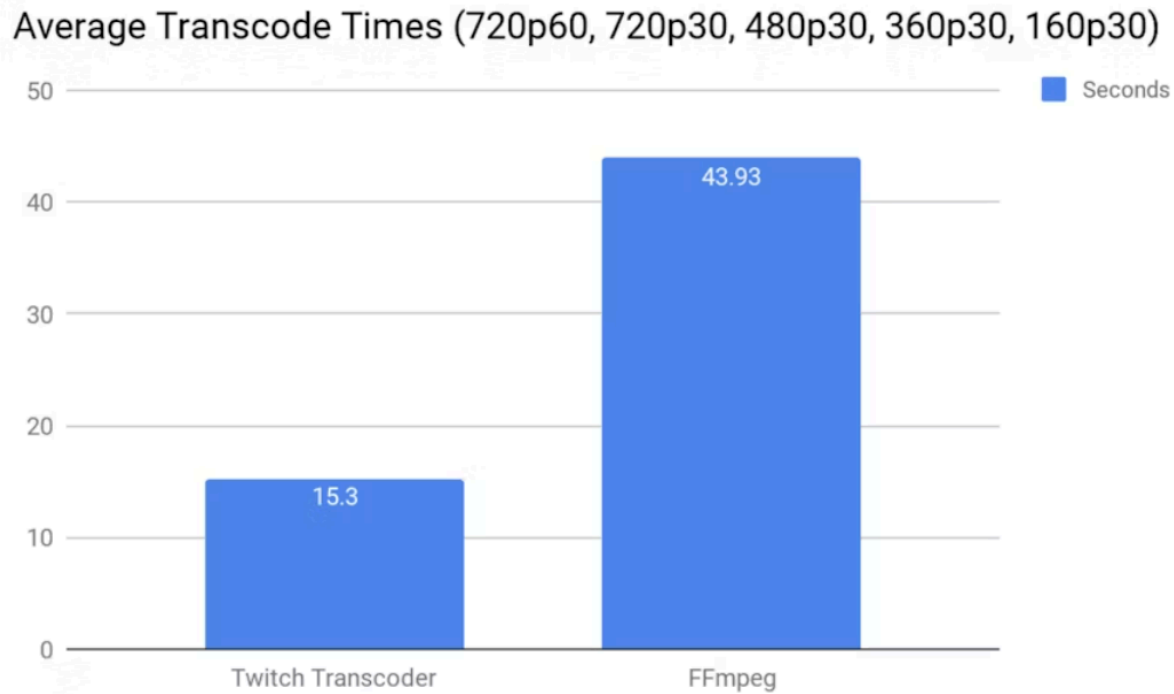
Note. This graphic represents the flow of input and output using the transcoder. The RTMP is processed in the transcoder, thus being converted into various formats simultaneously. Reprinted from TwitchBlog, Gong, J., Dhanju, S., Lu, C.-C., & Shen, Y., <https://blog.twitch.tv/en/2017/10/10/live-video-transmuxing-transcoding-f-fmpeg-vs-twitch-transcoder-part-i-489c1c125f28/>

TwitchTranscoder vs. FFmpeg: A performance study. Twitch engineers, Jeff Gong, Sahil Dhanju, Chih-Chiang Lu, and Yueshi Shen performed a benchmark study evaluating the difference in performance between the two models with results revealing critical differences in the context of live-streaming necessities. The results are as follows:

1. Execution Speed: The TwitchTranscoder was found to perform 65% faster than FFmpeg through the study of processing multiple types of video outputs at the same time as found by Gong et al. (2017). TwitchTranscoder uses a technique called *multithreading*, which allows for simultaneous task handling at the same time. In the case of generating multiple streams of different resolution qualities (e.g. 1080p, 720p, 480p), a system known as the *Adaptive Bitrate Ladder* ensures TwitchTranscoder delivers the best quality without buffering. FFmpeg processes each resolution one at a time, leading to its significantly slower performance.

Figure 4

TwitchTranscoder vs. FFmpeg Average Performance Times



Note. This is the resulting graph comparing the difference of transcoding times between the TwitchTranscoder and FFmpeg. Adapted from TwitchBlog, by J. Gong, et. al, <https://blog.twitch.tv/en/2017/10/23/live-video-transmuxing-transcoding-f-ffmpeg-vs-twitch-transcoder-part-ii-4973f475f8a3/>

2. Scalability: TwitchTranscoder is optimized to handle a large volume of simultaneous live streams with minimal overhead. *Minimal overhead* refers to the optimal use of hardware resources such as memory and CPU usage to maximize performance. The Transcoder shares tasks across multiple streams, saving redundancy in the workload and operating faster. In comparison, FFmpeg lacks the optimization for large-scale use and consumes high amounts of resources during execution.
3. Synchronization: The key to successful live streaming relies on seamless video quality adaptations in a viewer's internet speed. Precision over synchronizing video resolution is

necessary to avoid buffering and playback interruptions, which is exhibited in the performance of TwitchTranscoder. FFmpeg was not created for live-streaming purposes, meaning that it does not have smooth synchronization capabilities, causing buffering and stuttering to be much more likely.

Implications of Twitch's performance study. Twitch's focus on creating an optimal solution to the transcoding process is beneficial for all users, preserving smooth transitions and quality, while also cutting down on operational costs. TwitchTranscoder addresses the many unique challenges of developing a specialized live-streaming platform and sets the stage for future innovations.

Challenges in Achieving Compression and Low Latency

Despite Twitch's advanced innovations, factors such as device compatibility, high audience counts, and network instability create issues disturbing low latency. Older devices may struggle to process streams as effectively as newer devices, leading to playback issues. Popular streams can strain the server capacity, pushing an increase in latency. Additionally, unstable internet connections may introduce bandwidth fluctuations, causing delays for both creators and viewers.

Why Seamless Streaming Matters. Effective compression and latency management are critical for both users and the platform. For viewers, low latency enables real-time interactions fostering stronger connections to streamers. On the other side, streamers can engage with their audience, enhancing audience engagement and retention. For Twitch, these innovations provide a competitive edge in the live-streaming industry, allowing the platform to meet high user demands while maintaining a reputation for its technical strengths.

Conclusion

Twitch's technical infrastructure and innovations have solidified its position as a leader in the live-streaming industry. Twitch demonstrates a deep commitment to optimizing the experience for both content creators and its viewers. By combining cutting-edge technologies with a focus on user experience, Twitch optimizes the streaming experience for all users, setting a high standard for competitors. While there are many live streaming platforms available online, none can compete with the advanced technologies and innovations that Twitch has. Its advanced recommendation algorithm produces highly personalized content, creating engaging experiences for all users across the platform. Challenges such as transparency and potential biases favoring more popular creators highlight areas for future improvement. Twitch's compression techniques and latency management enable seamless real-time interactions while accommodating users with fluctuating network capabilities. These innovations reflect Twitch's ability to meet the immense demands of its global user base while scaling effectively to maintain high performance.

Live-streaming technology, as demonstrated by Twitch, is a revolutionary technology across industries beyond entertainment. It transforms our ability to communicate, learn, and interact with others. Some use cases for this include telehealth systems for healthcare, virtual classrooms for education, and long-distance communication for conferences and meetings. Twitch's technology allows it to be a strong role model for breaking geographical barriers to communication and building real-time connections from anywhere in the world.

Future innovations in advanced video codecs such as AV1 present opportunities to enhance streaming efficiency and support immersive virtual reality experiences. As a platform at the forefront of performance, scalability, and innovation, Twitch empowers millions of creators and viewers globally, preserving its position as a dominant leader in the live-streaming industry.

References

- Puri, R., Lafata, C., Pan, K., & Kwong, E. (2022, April 26). *Ingesting Live Video Streams at Global Scale*. Twitch.tv.
https://blog.twitch.tv/en/2022/04/26/ingesting-live-video-streams-at-global-scale/?utm_source=blog.quastor.org&utm_medium=referral&utm_campaign=how-twitch-processes-millions-of-video-streams
- Twitch Legal. (2024, February 15). *Twitch.tv - Recommendations On Twitch*. Twitch.tv.
<https://www.twitch.tv/p/en/legal/recommendations-on-twitch/>
- Gong, J., Dhanju, S., Lu, C.-C., & Shen, Y. (2017, October 10). *Live Video Transmuxing/Transcoding: FFmpeg vs TwitchTranscoder, Part I*. Twitch.tv.
<https://blog.twitch.tv/en/2017/10/10/live-video-transmuxing-transcoding-f-ffmpeg-vs-twitch-transcoder-part-i-489c1c125f28/>
- Gong, J., Dhanju, S., Lu, C.-C., & Shen, Y. (2017, October 23). *Live Video Transmuxing/Transcoding: FFmpeg vs TwitchTranscoder, Part II*. Twitch.tv.
<https://blog.twitch.tv/en/2017/10/23/live-video-transmuxing-transcoding-f-ffmpeg-vs-twitch-transcoder-part-ii-4973f475f8a3/>